

CoSc 3311

Fundamentals of Software Engineering

Dagmawi Lemma (PhD)

Assistant Professor, AAU

courses.dag@gmail.com

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

1

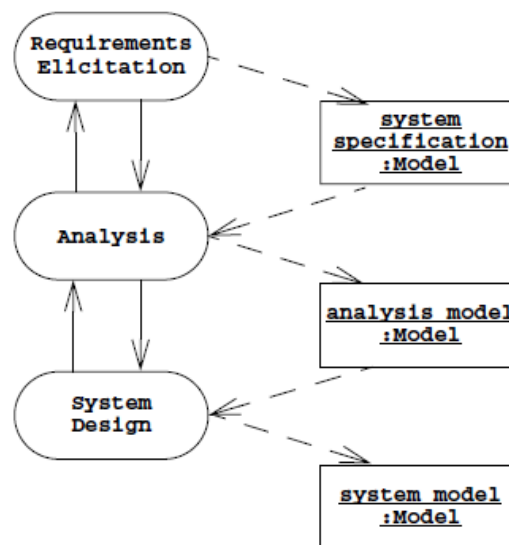
3. Requirements Analysis

CoSc3311 – Fundamentals of Software Engineering

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

119



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

120

Overview of Analysis

- The analysis model is composed of three individual models:
 - The functional model , represented by use-cases and scenarios,
 - Note that we have dealt with use-case and scenario during the elicitation as well; but those shall be refined further during the analysis phase
 - the dynamic model, represented by sequence diagrams, state-chart and activity diagram
 - The static model (object model), represented by class and object diagrams

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

121

Analysis Activity

- The order may vary and the process is iterative, but the followings are main activities during analysis
 - identifying entity, boundary and control classes/objects
 - identifying object attributes
 - This can be done at high level, we may not be particular to the visibility of the attributes
 - identifying object responsibilities and operations
 - identifying relationship among objects
 - modeling generalization relationships
 - modeling interactions between **objects**: *Sequence Diagrams*
 - *Can be further explained by Activity Diagram*
 - modeling objects/class behavior with state charts
 - reviewing the analysis model (verification and validation)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

122

Identifying Objects

- We have discussed
 - three main types of objects
 - that the objects need to be identified first from each use-case (especially the entity and boundary classes/objects)
- During the object identification, *mainly* we have to identify the followings, which are also model components
 - name
 - attributes
 - operation that can be performed by/on the object
 - type of the object/class (entity, control, boundary)
 - Differentiate object Vs class
 - E.g.
 - web page → class
 - Reservation Page → object

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

123

Abbott's heuristics for mapping parts of speech to model components

Part of speech	Model component	Examples
Proper noun	Object	Alemu
Common noun	Class	Library member
Action verb	Operation	reserve, release, open, create, count, submit ...
Being verb	Inheritance	Text book is a book
Having verb	Aggregation / composition	Reservation list contains list of tuple made up of date, time, member ID and ISBN
Adjective	Attribute	book title, book author, member name,

R. Abbott, "Program design by informal English descriptions," *Communications of the ACM*, Vol. 26, No. 11, 1983.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

124

Modeling Relationships Between Objects: Class Diagrams

- The structure of the software/system can be modeled using class diagram, which describes the system in terms of the relationship between objects
- The relationship between classes/objects is represented using links that designates the relationship as
 - Association
 - Generalization/Specialization
 - Aggregation
 - Composition

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

128

Modeling Interactions Between Objects: Sequence Diagrams

- Once objects/classes those are participating in a use-case are identified, it is important to model their interaction to depict how the behavior of a use-case (or scenario) is distributed among its participating objects
 - Each use-case need to be distinctly modeled using sequence diagram
- Note that sequence diagrams are usually not a good medium for communication with the users/client
 - But the intention of the analysis is to specify the system from the developers' point of view

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

129

Modeling Interactions Between Objects: Sequence Diagrams

- Sequence diagrams allow the developers to
 - identify missing objects
 - clarify grey areas in the system specification
 - identify missing attributes and operations (messages)
 - In particular, control objects/classes are identified during interaction model
 - As recommendation, at least one control object for each use-case
 - Note that having more control object might lead to complex model; yet we want to have simplification

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

130

Modeling Interactions Between Objects: Sequence Diagrams

- Each use-case need to have a corresponding sequence diagram so as to capture the communication between the objects, which are identified in the respective use-case
- The communications between the objects can appear as either of the three types of message
 - Asynchronous
 - The sender of the message is not expected to wait for other action before sending the next message
 - Represented by an open arrow
 - Synchronous
 - The sender of the message is expected to wait for other action before sending the next message
 - Represented by an open arrow
 - Return
 - Represents a return value
 - Represented by dashed arrow

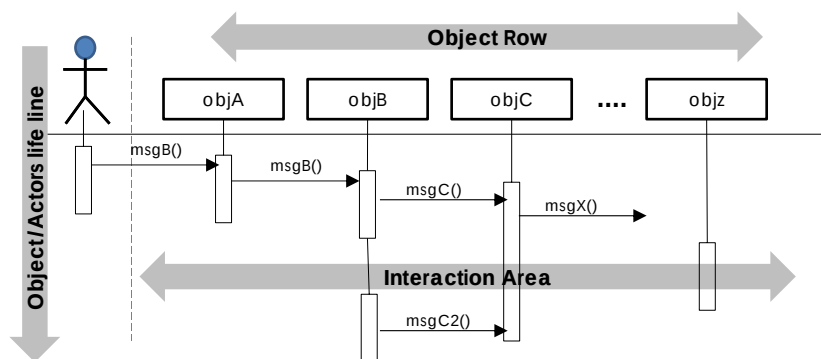
Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

131

Modeling Interactions Between Objects: Sequence Diagrams

- Sequence diagram can have two **major** parts
 - Object row and
 - Interaction or object-life-line area, where the life of object is drawn downward



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

132

Activity Diagram

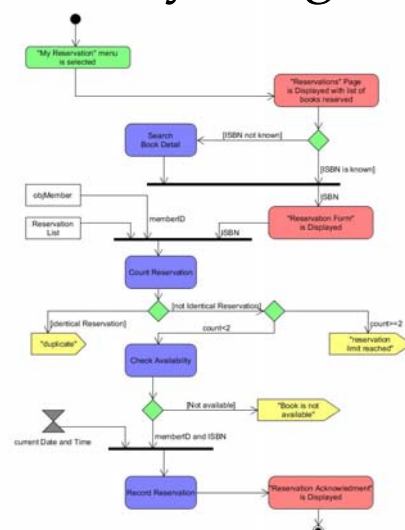
- Sometimes in a sequence diagram, it could be complicated to understand the flow of interaction (especially when several fragments exist).
- In order to further clarify the interaction in a use-case, activity diagram can be used.
- Activity diagram is also used to describe how activities are coordinated to provide a service which can be at different level of abstraction.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

135

Activity Diagram Example



For more information about
Activity diagram you may visit
<https://www.softwareideas.net/uml-activity-diagram>

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

136

State Machine Diagram

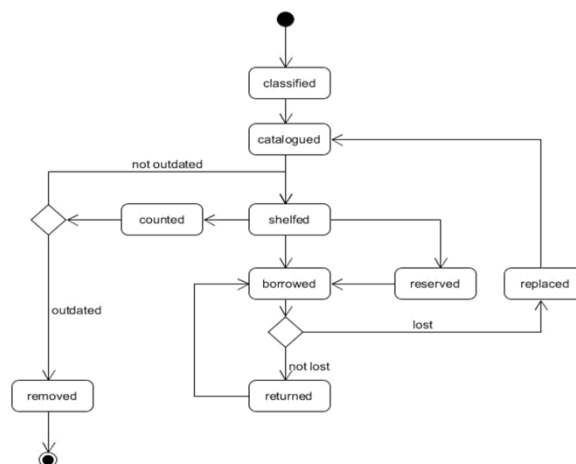
- Used to model dynamics of a particular class/object
- What are the various states that an object pass through? Perhaps across various use case
- If we answer this question then it is possible to capture the property and activity of an object

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

137

State Machine Diagram Example



Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

138

Sequence Diagram Vs State Diagram

- Sequence Diagram
 - Captures requirements by analyzing the interaction of many objects in a single use case.
- State Diagram
 - Captures requirements by analyzing the property (state) of an object in multiple use case.

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

139

Summary of Requirements Analysis

- Requirements Analysis aims to produce the System Specification regardless of any specific technology or solution
- Unlike the System Specification we have during requirements elicitation, requirements analysis produce set of models which is readable by software engineers and technical peoples (designers, programmers, analysts, etc.)

Dagmawi L.
(courses.dag@gmail.com)

CoSc3311 - Fundamentals of Software
Engineering

140

Summary ...

Elicitation	Analysis
- Scenarios are used to describe events in the system - Use-cases are described by aggregating scenarios as applicable - Relationship between use-cases are identified - Functional and nonfunctional requirements are extracted from the use-cases	Use-case diagram is used to model the entire system functionality - Each use-case is mapped to sequence diagram to capture more requirements and additional methods in the classes by showing the interaction between the objects – focus is on <u>the interactional relationship</u> between object in <u>a particular use-case</u> - Sequence diagrams might be further described by activity diagram
Actors are identified and described how they are participating in a use-case	Actors appear in the sequence diagram and their role is validated and verified
Class/Objects are identified and together with their property, methods and responsibilities	- Static part of the system is modeled using class diagram showing the various possible <u>structural relationship</u> between the classes/objects <u>across the system</u> - Dynamic behavior of each class across several use-cases is captured and modeled using state chart (state machine) diagram
The specification shall be validated and verified iteratively	The models shall be validated and verified iteratively